

CS & IT ENGINEERING



Computer Network - 2

Transport & Application Layer

Lecture No. - 02

By - Abhishek Sir





Recap of Previous Lecture



Topic

DHCP

Topic

Socket Programming





Topics to be Covered



Topic

Socket Programming

Topic

TCP Options

Topic

TCP Window Scale

ABOUT ME



Hello, I'm **Abhishek**

- GATE CS AIR - 96
- M.Tech (CS) - IIT Kharagpur
- 12 years of GATE CS teaching experience

Telegram Link : https://t.me/abhisheksirCS_PW





Topic : Creating a Socket

(-1) → 2's comp. form
[Binary all one's]



```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
int socket ( int domain, int type, int protocol );
```

domain : Protocol Family
[PF_INET, AF_INET, PF_UNIX etc]

type : Communication Protocol semantics
[SOCK_STREAM, SOCK_DGRAM]

protocol : Specify particular protocol, set "zero" as default

return : On success, a file descriptor for the new socket
On error, -1 is returned

Success → file descriptor
Error → -1

→ sock_fd



Topic : Bind a name to a Socket

⇒ Server IP & Port
Binding to Welcome
Socket



```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
int bind ( int sockfd, const struct sockaddr *addr, socklen_t addrlen );
```

sockfd : file descriptor of the socket (welcome socket)

return : On success, zero
On error, -1 is returned

Success → 0
Error → -1

⇒ Bind is optional for client;



Topic : Port Number

$\Rightarrow 16 \text{ bit} / 2 \text{ byte}$



$\rightarrow$ Port numbers are divided into three ranges :

HTTP $\rightarrow$ 80
SMTP $\rightarrow$ 25
DHCP $\rightarrow$ 67/68
FTP $\rightarrow$ 20/21

1. System (Well-known) Ports :
[assigned and controlled]

0 to 1023

* Mostly
for Servers

2. Registered Ports : [1024 to 49151]
[not assigned or controlled, but can be registered to prevent duplication]

3. Dynamic (Private) Ports : [49152 to 65535]
[not assigned, controlled or registered]

* Short term use, Mostly for clients



Topic : Listen for connections



```
#include <sys/socket.h>
```

```
int listen ( int sockfd, int backlog );
```

sockfd : file descriptor of the socket (welcome socket)

backlog : maximum length for queue of pending connections

return : On success, zero
On error, -1 is returned

→ Connection Req. allowed to enter in accept queue,

→ Connection Request blocked;
(Accept queue is full)



Topic : Socket Programming with TCP



P₂

TCP Server

socket() → Create Socket
(Welcome socket)
(Active Socket)

bind() ← local Address
binding

listen() ← Ready to Read
(Passive Socket)

[MCQ] [GATE-2014, Set-2, 1-Marks] (IIT-KGP)



#Q. Which one of the following socket API functions converts an unconnected active TCP socket into a passive socket?

☐ A Connect

☒ B Bind

☐ C Listen

☐ D Accept

Ans: C



Topic : Initiate a connection



```
#include <sys/socket.h>
```

```
int connect ( int sockfd, const struct sockaddr *addr, socklen_t addrlen );
```

sockfd : file descriptor of the socket at client

return : If connection succeeds, zero
On error, -1 is returned

→ After RTT

→ After time-out

⇒ ISN chooses by connect at client

⇒ At time of connection Request,
OS assign dynamic Port No. (if no bind used at client)



Topic : Socket Programming with TCP



P₁

TCP Client

create
Socket → socket()

connect()

return 0;
success;
Established

SYN connection Req.

SYN+ACK, connection
Accept

FINAL-ACK
No Payload

P₂

TCP Server

socket()

bind() (Welcome Socket)

listen() → Ready to
Listen

return 0;
conn. Req. is in
Accept queue

[MCQ] [GATE-CS-2008, 1-Marks]

1155

(2 Mark)



#Q. Which of the following system calls results in the sending of SYN packets?

- ☒ A Socket
- ☒ B Bind
- ☒ C Listen
- ☐ D Connect

Ans: D



Topic : Socket Programming with TCP



$P_1 = P$

TCP Client

create
socket $\leftarrow$ socket() ✓

connect()

SYN connection Req.

wait for (SYN+ACK)
till time-out;

Return -1;
(Error)

TO

$P_2 = S$

TCP Server

✓ socket()

✓ bind() (Welcome socket)

✓ listen() $\rightarrow$ Ready to Listen

return -1;
conn. Req. blocked;

[MCQ] [GATE-CS-2008, 2-Marks]

II SC



#Q. A client process P needs to make a TCP connection to a server process S. Consider the following situation: the server process S executes a socket(), a bind() and a listen() system call in that order, following which it is preempted. Subsequently, the client process P executes a socket() system call followed by connect() system call to connect to the server process S. The server process has not executed any accept() system call. Which one of the following events could take place?

- ☒ A connect() system call returns successfully (return 0) $\Rightarrow$ FALSE
- ☒ B connect() system call blocks $\Rightarrow$ connect Request blocked at server
- ☒ C connect() system call returns an error (return -1) $\Rightarrow$ TRUE
- ☒ D connect() system call results in a core dump

Ans: C



Topic : Accept a connection



→ Creates a new connected (connection) socket

⇒ Pick conn. Req. from accept queue

⇒ Welcome Socket make request to OS for new socket;

```
#include <sys/socket.h>
```

```
int accept ( int sockfd, struct sockaddr *_Nullable restrict addr,  
             socklen_t *_Nullable restrict addrlen );
```

sockfd : file descriptor of the socket (welcome socket)

return : On success, file descriptor for the accepted socket
On error, -1 is returned

→ sock_fd_new



Topic : Socket Programming with TCP



TCP Client

socket()

connect()

TCP Server

socket()

bind()

listen() $\Rightarrow$ Welcome Socket

$\leftarrow$ (Queue)

accept() $\Rightarrow$ create new socket

ESTB *

(SYN + ACK)

FINAL-ACK

Welcome Socket

Estb

Connection Socket Active during Established

[MCQ] [GATE-2015, Set-2, 1-Marks]

(IIT-K)



#Q. Identify the correct order in which a server process must invoke the function calls accept, bind, listen, and recv according to UNIX socket API.

- A listen, accept, bind, recv → read
- ✓ B bind, listen, accept, recv
- C bind, accept, listen, recv
- D accept, listen, bind, recv

Ans: B



Topic : Write to a socket

sockfd → socket

- (1) Client Address;
- (2) Server Address;



```
#include <unistd.h>
```

```
ssize_t write ( int sockfd, const void buf[.count], size_t count);
```

Annotations: An arrow points from 'write' to 'send'. A bracket under 'buf[.count]' points to 'send'. A bracket under 'count' points to 'send'.

sockfd : file descriptor of the socket (conn. socket)

return : On success, the number of bytes written is returned
On error, -1 is returned

⇒ Write & Read (Send & Recv) is used in case of connected socket; (Like TCP, during data transfer)



Topic : Read from a socket



```
#include <unistd.h>
```

```
ssize_t read ( int sockfd, void buf[.count], size_t count);
```

sockfd : file descriptor of the socket (conn. socket)

return : On success, the number of bytes read is returned
On error, -1 is returned



Topic : Socket Programming with TCP



TCP Client

socket()

connect()

write()

read()

TCP Server

socket()

bind()

listen()

accept()

Full Duplex Data Transfer

→ read()

write()



Topic : Close a socket



```
#include <unistd.h>
```

```
ssize_t close ( int sockfd );
```

sockfd : file descriptor of the socket (conn. socket)

return : On success, returns zero
On error, -1 is returned

→ After RTT

↑ After Time-out



Topic : Socket Programming with TCP



TCP Client

socket()

connect()

write()

read()

close()

TCP Server

socket()

bind()

listen()

accept()

read()

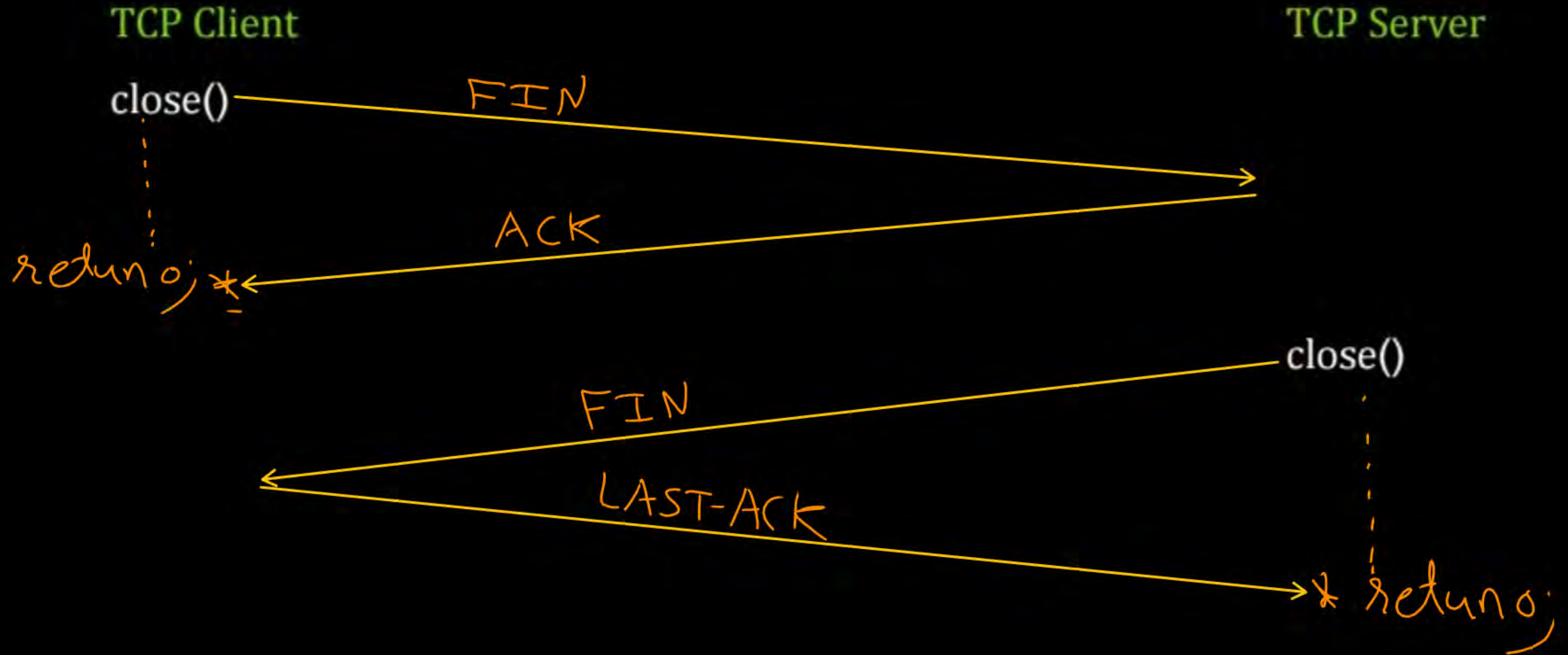
write()

close()

4-way handshake



Topic : Socket Programming with TCP





Topic : Server Type



→ Several clients can request service of the server in the same time.

Non-concurrent Server :

- Service one client at one time
- Other client request must wait

⇒ old;

Concurrent Server :

- Services all client requests simultaneously

⇒ New;



Topic : Send a message on socket

```
#include <sys/socket.h>
```

```
ssize_t sendto ( int socket, const void *message, size_t length, int flags,  
                const struct sockaddr *dest_addr, socklen_t dest_len );
```

sockfd : file descriptor of the socket (UDP socket)

return : On success, the number of bytes send is returned
On error, -1 is returned

* sendto & recvfrom is used in case of unconnected socket;
Mostly in UDP sockets;



Topic : Receive a message from a socket



```
#include <sys/socket.h>
```

```
ssize_t recvfrom ( int socket, void *buffer, size_t length, int flags,  
                  struct sockaddr *address, socklen_t address_len );
```

sockfd : file descriptor of the socket (UDP socket)

return : On success, the number of bytes receive is returned
On error, -1 is returned



Topic : Socket Programming with UDP



Unconnected,

UDP Client

socket() ✓

sendto()

recvfrom()

close()

delete;

UDP Server

socket() ✓

bind() ✓

recvfrom()

sendto()

close()

Never

UDP segment

UDP segment

while loop



Topic : Connected UDP sockets

⇒ Not like TCP connected socket



UDP Client

socket()

connect()

write()

read()

close()

UDP Server

socket()

bind()

read()

write()

close()

connect

while loop

Loop

[MCQ] [GATE-2017, Set-2, 2-Marks]

IIT-R, [H.W.]



#Q. Consider socket API on a Linux machine that supports connected UDP sockets. A connected UDP socket is a UDP socket on which connect function has already been called. Which of the following statements is/are CORRECT?

- I. A connected UDP socket can be used to communicate with multiple peers simultaneously.
- II. A process can successfully call connect function again for an already connected UDP socket.

- ☐ A I only
- ☐ B II only
- ☐ C Both I and II
- ☐ D Neither I nor II



2 mins Summary



Topic

Socket Programming ✓

Topic

~~TCP Options~~

Topic

~~TCP Window Scale~~

TCP Times



THANK - YOU

